浙江大学

本科实验报告

课程名称：电子电路设计II

姓　　名：

学　　院：信息与电子工程学院

　　系：

专　　业：信息工程

学　　号：

指导教师：施红军、李锡华、叶险峰

2025 年 4 月 28 日

专业：信息工程

姓名：

学号：

日期：2025.4.28

浙江大学实验报告

课程名称：电子电路设计实验II

指导老师：施红军、李锡华、叶险峰

成绩：

实验名称：电流电压转换电路设计与多功能数字时钟设计

一、实验目的

1．了熟悉电流信号转换成电压信号的原理，掌握标准电流信号转换成电压信号设计方法。

2．学习掌握用 Arduino UNO 设计数字时钟，掌握 Arduino UNO 扩展板的设计与制作；

3．学习掌握 PCB 电路板的设计和制作；

二、实验任务

2.1 电流电压转换电路设计

1．熟悉电流-电压转换电路的工作原理。

2．设计要求：将标准电流信号 4mA~20mA 转换为标准电压信号 0V~10V，误差控制在5%以内。

2.2 多功能数字时钟设计

1．熟设计并制作一个能够显示时间、日期和星期的多功能数字时钟。

2．实现闹钟功能，当达到设定时间时，能够通过蜂鸣器或 LED 进行提示。

3．附加功能：实现温度显示功能，通过温度传感器实时显示当前环境温度。

4．编写相应的 Arduino 程序，实现数字时钟的各项功能。

三、主要仪器设备

3.1 电流电压转换电路设计

电阻、电容等辅助元件，运算放大器，电脑，恒流源等

3.2 多功能数字时钟设计

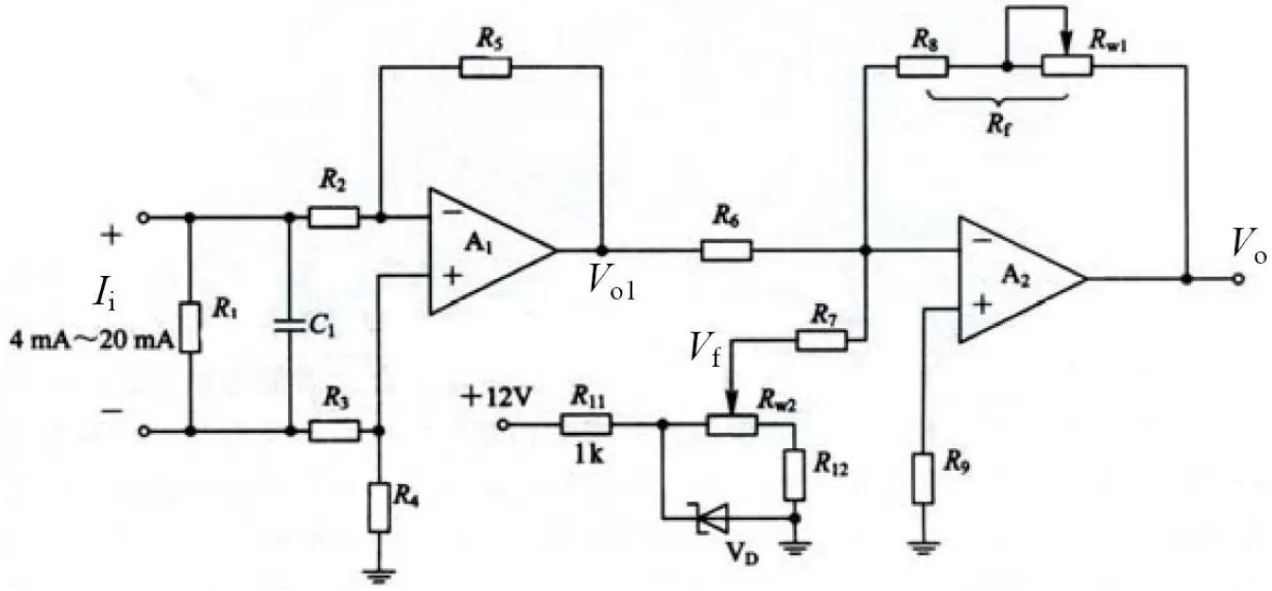
Arduino 板（如 Arduino Uno），显示屏（OLED 或 LCD），时钟模块 DS1302，蜂鸣器，LED，温度传感器（如 LM35），电阻、电容等辅助元件，电脑及 Arduino IDE 软件等

四、实验原理与设计

4.1 电流电压转换电路设计

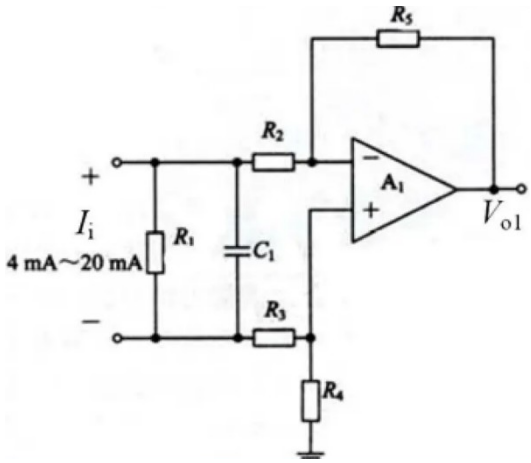
1．实验原理：自动控制技术中，变送器输出的 4~20mA 标准电流信号通常需要转换成 0~10V 的电压信号后进行进一步处理。4mA 为满量程的 0%，对应输出电压的 0V，12mA 为满量程的 50%，对应输出电压的 5V，20mA 为满量程的 100%，对应输出电压的 10V。

2．总体电路设计：电流电压转换电路由两级放大电路构成，第一级放大电路将输入电流 4~20mA 转换为电压-0.8~-4V，第二级放大电路将输入电压-0.8~-4V 转换为 0~10V



（1）电阻 R1 跨接在电流源两端，对从上向下流经的 4 ~ 20mA 输入电流 Ii 进行取样。

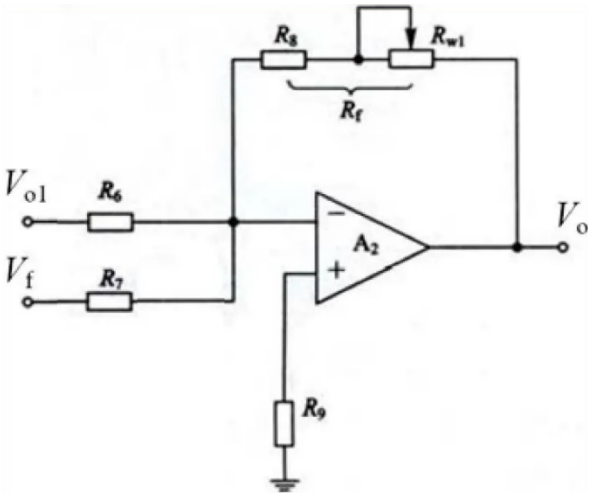
（2）第一级电路：运放 A1 与电阻构成差动放大器，当 R2、R3、R4、R5 相等时，Vo1=-R2Ii。4 个匹配电阻 R2、R3、R4、R5 可以预先取定，取为常用中等阻值 10kΩ。电阻 R1 采用变送器标准负载：阻值 250Ω、精度 0.1%，选用额定功率为 1/4W 的电阻



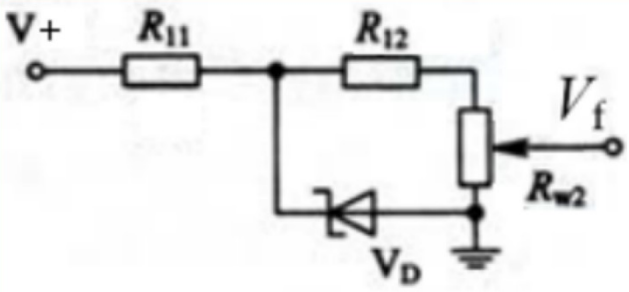
（3）第二级电路：反相加法器电路，实现从 Vo1 的 -（4 ~20mA）×R1 到 Vo 的 0~10V 的电平变换。只要合理选取 R6、R7 的阻值，调整 Vf 和 Rf，可以产生线性变化。

$$V_o=\frac{R_f}{R_6}I_iR_1-\frac{R_f}{R_7}V_f$$

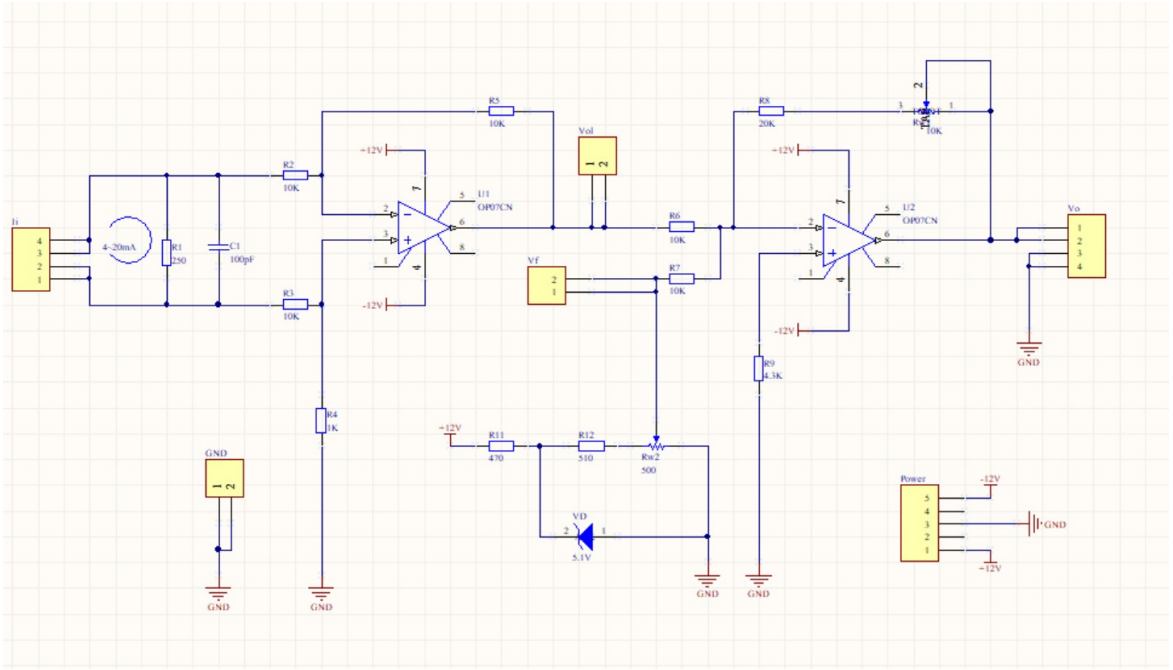
当 Vo1 从 - 1V 减小到 - 5V、变化 4V 时，Vo 需要从 0V 增加到 10V、变化 10V。从 Vo1~Vo 变换式可知，应使 Rf 等于 2.5 倍的 R6。若取 R7 = R6 =10kΩ，为了使得 Ii = 4 mA 、Vo1 = - 1V 时，Vo = 0V，应把 Vf 调为 +1V。



（4）利用稳压管使Vf产生稳定电压：分压后要产生 1V 左右的可调电压，分压电路可由上固定电阻串联下电位器构成。取流过分压电路电流为 5mA，则分压电路总电阻约为 1kΩ。取上固定电阻 R12=510Ω，下电位器 Rw2 = 500Ω，使得分压 Vf 将可在 0~2.5V 之间可调。



3．原理图：



（1）元件封装选择：OP07 为 DIP8，1/4W 电阻为 AXIAL0.4，瓷片电容为 RAD0.2，稳压二极管为 DIODE0.4，电位器为 VR5。

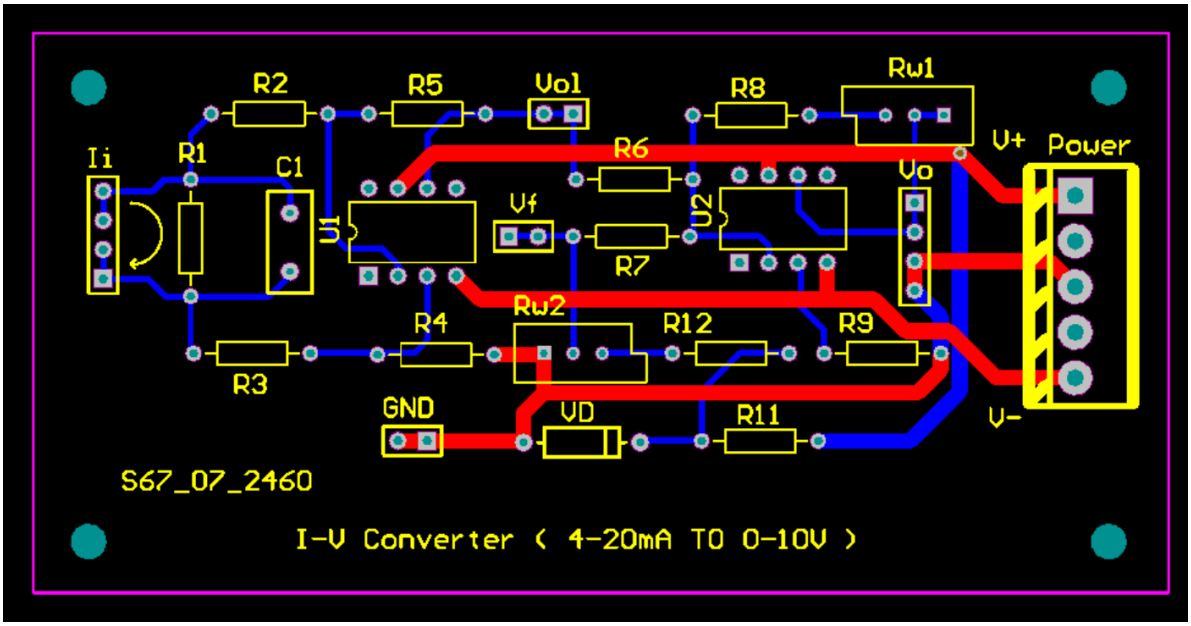
（2）接插件选择与封装：电源接口、输入输出接口、关键测试点、参考地测试点等。

①本设计中电源接口符号用 5 引脚的 Header5（封装 PowerPlug5）

②输入接插件用的是 4 引脚排针（封装 SIP4）

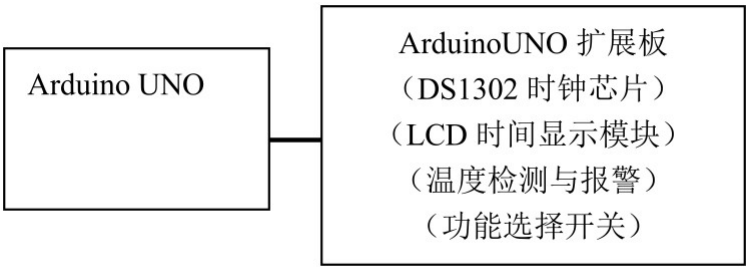
③图中的 Vo1、Vf 和 GND 接口分别为关键测试点和参考地测试点，封装都为 SIP2 的 2 引脚排针。

4．PCB 设计图：



4.2 多功能数字时钟设计

1．硬件框图

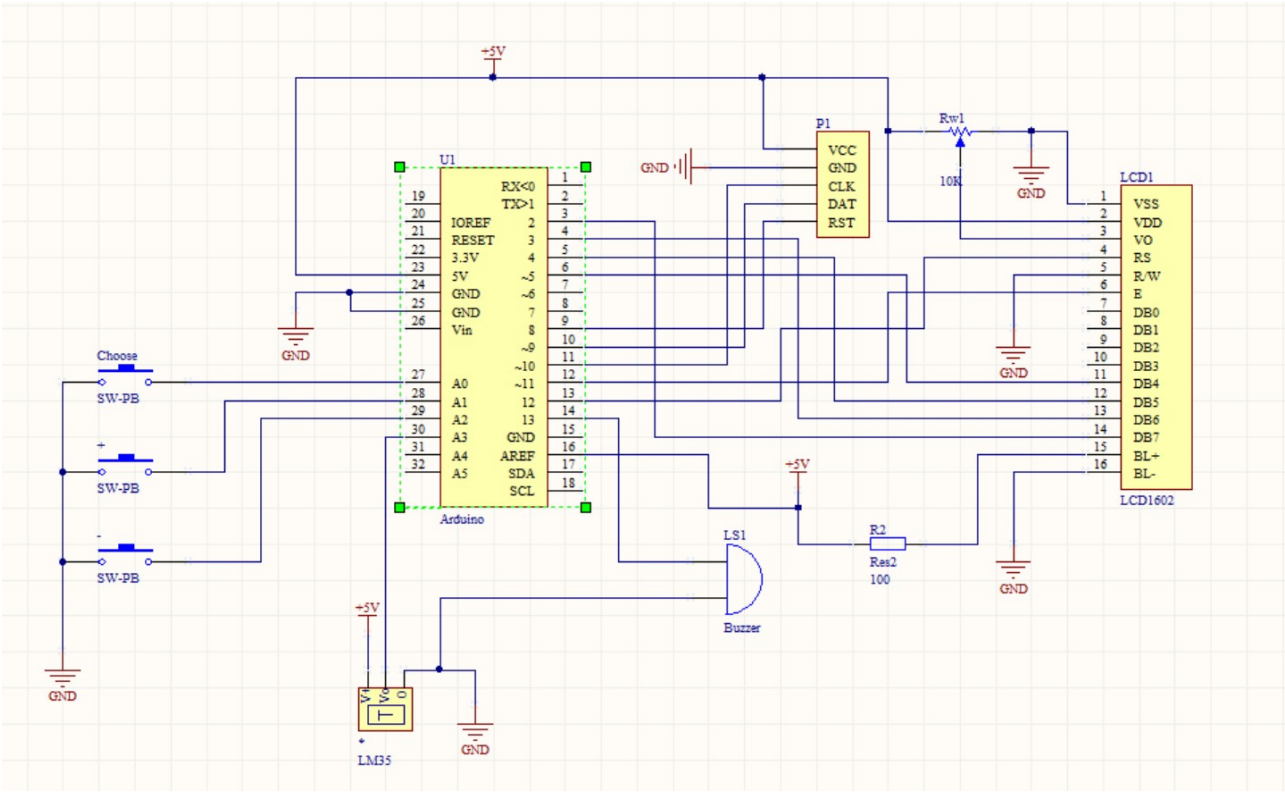


（1）DS1302 时钟芯片：附加 31 字节静态 RAM，采用 SPI 三线接口与 CPU 进行同步通信，并可采用突发方式一次传送多个字节的时钟信号和 RAM 数据。工作电压宽达 2.5～5.5V。采用双电源供电。

（2）LCD 显示模块：使用 Arduino UNO 直接驱动 1602 液晶显示字母和数字，LCD 显示模块有一个优势就是将驱动电路集成在模块中。

（3）温度传感器：采用 LM35

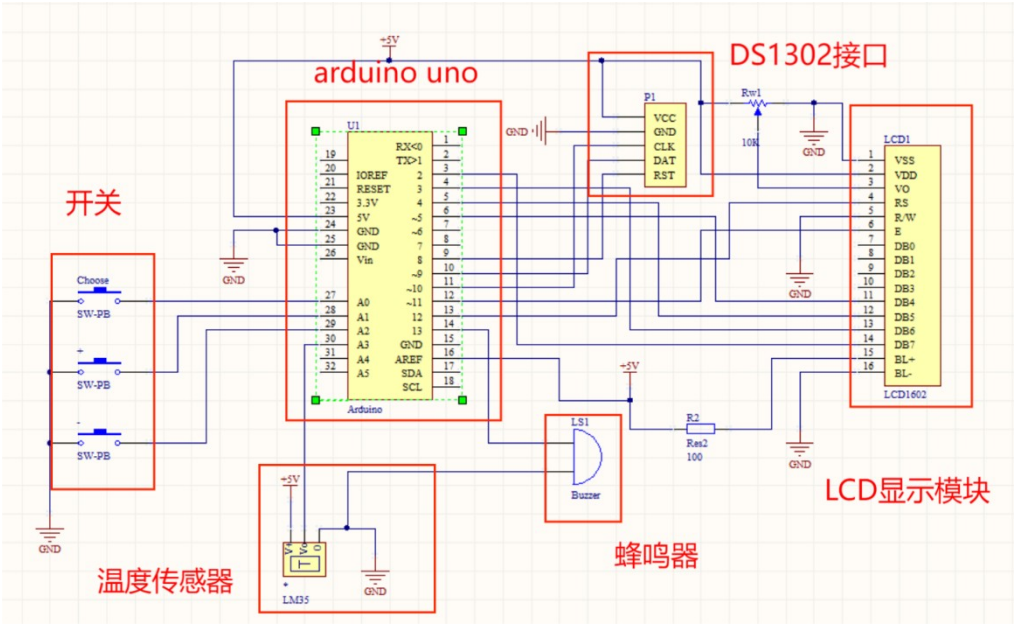
2．原理图：



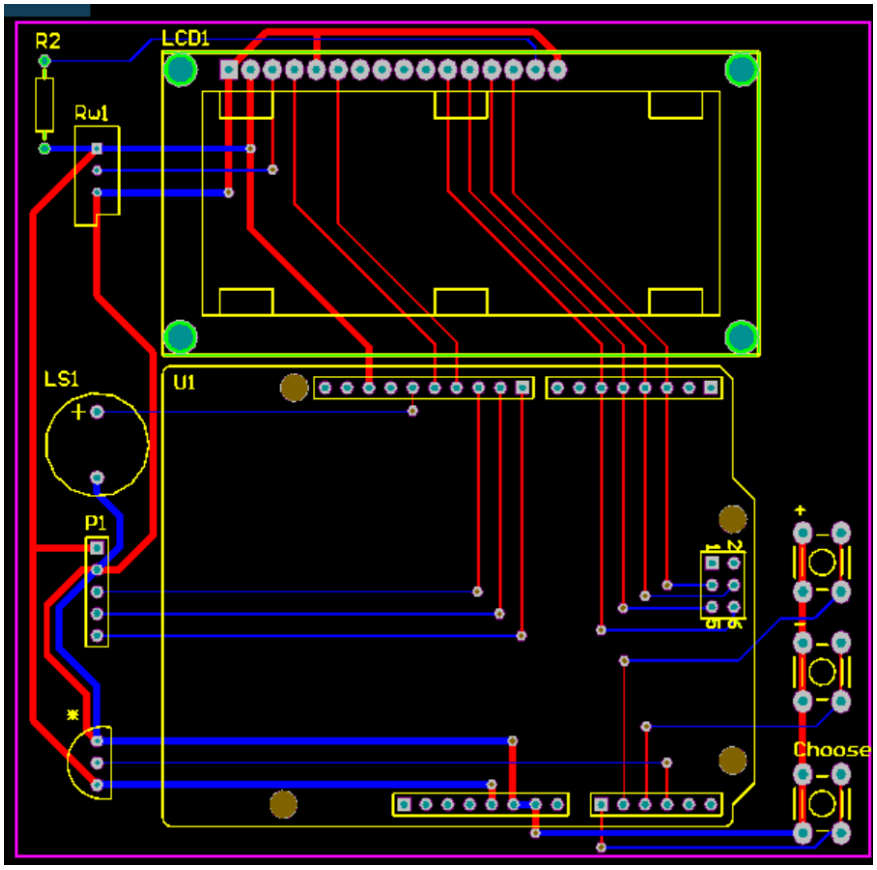
（1）根据框图放置元件，合理布局，并选择合适的封装：LCD 为 LCD1602，温度传感器为 LM35，蜂鸣器为 Buzzer，SW 开关为 AN-S 小按钮开关，1/4W 电阻为 AXIAL0.4

（2）接插件选择为 P1 接 DS1302 时钟芯片（封装为 SIP5）

（3）模块介绍：



3．PCB 设计图：

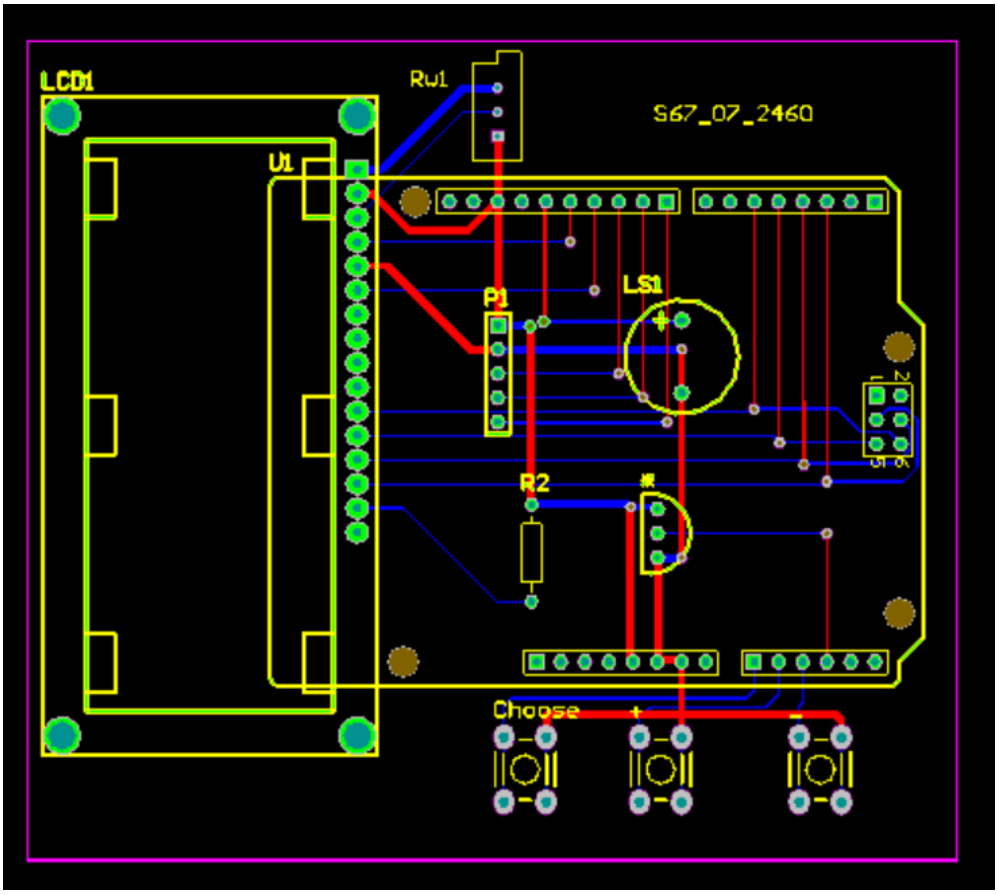


（1）根据框图放置元件，合理布局，并选择合适的封装：LCD 为 LCD1602，温度传感器为 LM35，蜂鸣器为 Buzzer，SW 开关为 AN-S 小按钮开关，1/4W 电阻为 AXIAL0.4

（2）在绘制 pcb 图时需要注意：

①器件之间的距离，一开始我连接的时候为了节省材料，发现 arduino uno 板只需要排针和最后底座相连接即可，所以将 LCD 等器件放在了 uno 板上的空白处（如下图所示），但是导致 pcb 报错，询问老师后得知，由于 ad 软件的规则限制，使得元器件不能重叠，而且 LCD 焊盘也违反了 ad 的规则，因此不必理会，但是我还是将 pcb 重新进行了规划得到了上述的 pcb 图

②Pcb 布线过程中发现，对于 LCD 的几个焊盘之间也有飞线，但是由于 LCD 焊盘在一条直线上，导致飞线覆盖了所有焊盘，导致布线时产生差错，仔细观察后可以看出只有部分焊盘需要布线



4．编写相应的 Arduino 程序，实现数字时钟的各项功能。

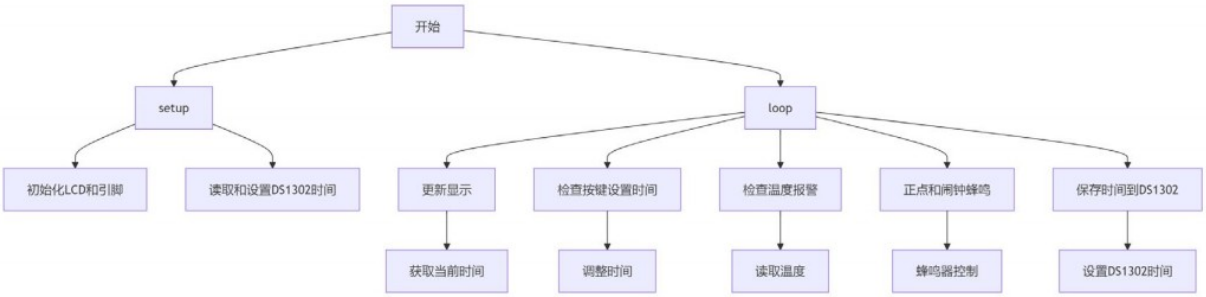
（1）流程图：

①开始：程序的入口点；

②setup()：初始化程序运行环境，包括 LCD 显示器和引脚模式；

③loop()：主循环，程序持续运行，包括更新显示，检查按键设置时间，检查温度报警，正点和闹钟蜂鸣，保存时间到 DS1302

流程图提供了代码执行的主要步骤，有助于快速理解代码的结构和逻辑。每个步骤都对应代码中的一个操作或函数调用，显示了程序如何读取输入、处理数据并输出结果。



（2）主要代码：（从上到下，从左到右）

```
/* *

* LCD RS pin to digital pin 12

* LCD Enable pin to digital pin 11

* LCD D4 pin to digital pin 5

* LCD D5 pin to digital pin 4

* LCD D6 pin to digital pin 3

* LCD D7 pin to digital pin 2

* LCD R/W pin to ground

* LCD VSS pin to ground

* LCD VCC pin to 5V

 * */

#include <DS1302.h>
```

```
    lcd.setCursor(9, 1);

lcd.print("SetTime");

            while(1){

                if(digitalRead(choose) == LOW)

{

                    delay(ButtonDelay);

                if(digitalRead(choose) ==LOW)

{

                        chose++;

                    }

                }
```

```
#include <LiquidCrystal.h>          //LCD1602 显示头文件

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

#define choose   A0    //选择端口
#define add   A1       //加
#define minus   A2     //减
#define Tone 13        //蜂鸣器端口
uint8_t CE_PIN   = 8;   //DS1302 RST 端口
uint8_t IO_PIN   = 9;   //DS1302 DAT 端口
uint8_t SCLK_PIN = 10;  //DS1302 CLK 端口
DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN);  //创建 DS1302 对象
unsigned long seconds;
int s = 0, m = 0, h = 0, d = 0, mon = 0, y =
        seconds = millis()/1000;
        Display();
        if(chose == 1){
            Set_Time(1, 1, HOUR);
//SetHour
        }else if(chose == 2){
            Set_Time(4, 1, MINUTE);
//SetMinute
        }else if(chose == 3){
            Set_Time(7, 1, SECOND);
//SetSecond
        }else if(chose == 4){
            Set_Time(9, 0, DAY);
//SetDay
        }else if(chose == 5){
          Set_Time(6, 0, MONTH);      //
```

```
0;     //时间进位
int second = 0, minute = 0, hour = 0, day = 0,
month = 0, year = 0;   //当前时间
int SECOND = 0, MINUTE = 0, HOUR = 0,
DAY = 0, MONTH = 0, YEAR = 0;   //初始时
间
int chose = 0, alarm_choose = 0 ,ButtonDelay =
20, frequence = 2093;
int alarm_hour = 7, alarm_minute = 30,
alarm_second = 0;   //闹钟时间
float Temperatures, Temp_Alarm = 30 ;


/** 格式化输出 */
void FormatDisplay(int col, int row,int num){
      lcd.setCursor(col, row);
      if(num < 10)      lcd.print("0");
```

```
SetMonth
            }else if(chose == 6){
                  Set_Time(3, 0, YEAR);
//SetYear
            }else if(chose >= 7) {
                  chose = 0;
                  break;
            }
        }
        delay(150);//新增（2024）
    }
}


/** 设置闹钟小时 */
void Set_Alarm_Hour(){
    DisplayCursor(1, 1);
    if(digitalRead(add) == LOW){
```

```
        lcd.print(num);
}

/** 计算时间 */
void time() {
        second = (SECOND + seconds) % 60;    //计算秒
        m = (SECOND + seconds) / 60;          //分钟进位
        FormatDisplay(6,1,second);

        minute = (MINUTE + m) % 60;    //计算分钟
        h = (MINUTE + m) / 60;          //小时进位
        FormatDisplay(3,1,minute);
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            alarm_hour ++;
            if(alarm_hour == 24){
                alarm_hour = 0;
            }
            FormatDisplay(0,1,alarm_hour);
        }
    }
    if(digitalRead(minus) == LOW){
        delay(ButtonDelay);
        if(digitalRead(minus) == LOW){
            alarm_hour --;
            if(alarm_hour == -1){
                alarm_hour = 23;
            }
            FormatDisplay(0,1,alarm_hour)
```

```
    hour = (HOUR + h) % 24;      //计算小时
    d = (HOUR + h) / 24;         //天数进位
    FormatDisplay(0,1,hour);

    lcd.setCursor(2, 1);
    lcd.print(":");
    lcd.setCursor(5, 1);
    lcd.print(":");
}

/** 根据年月计算当月天数 */
int Days(int year, int month){
    int days = 0;
    if (month != 2){
        switch(month){
            case 1: case 3: case 5: case 7: case 8:
;
                }
        }
}

/** 设置闹钟分钟 */
void Set_Alarm_Minute(){
    DisplayCursor(4, 1);
    if(digitalRead(add) == LOW) {
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            alarm_minute ++;
            if(alarm_minute == 60){
                alarm_minute = 0;

            }
```

```
case 10: case 12: days = 31;    break;
            case 4: case 6: case 9: case 11:    days =
30;    break;
        }
    }else{      //闰年
        if(year % 4 == 0 && year % 100 != 0 ||
year % 400 == 0){
            days = 29;
        }
        else{
            days = 28;
        }
    }
    return days;
}

/** 计算当月天数 */
                FormatDisplay(3,1,alarm_minute)
;
            }
        }
        if(digitalRead(minus) == LOW){
            delay(ButtonDelay);
            if(digitalRead(minus) == LOW){
                alarm_minute --;
                if(alarm_minute == -1){
                    alarm_minute = 59;
                }
            FormatDisplay(3,1,alarm_minute)
;
            }
        }
}
```

```
void Day(){
    int days = Days(year,month);
    int days_up;
    if(month == 1){
        days_up = Days(year - 1, 12);
    }
    else{
        days_up = Days(year, month - 1);
    }
    day = (DAY + d) % days;
    if(day == 0){
        day = days;
    }
    if((DAY + d) == days + 1 ){
        DAY -= days;
        mon++;
    }
```

```
/** 设置报警温度 */
void Set_Alarm_Temp(){
    DisplayCursor(10, 1);
    if(digitalRead(add) == LOW) {
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            Temp_Alarm ++;
        }
    }
    if(digitalRead(minus) == LOW){
        delay(ButtonDelay);
        if(digitalRead(minus) == LOW){
            Temp_Alarm --;
        }
    }
}
```

```
    if((DAY + d) == 0){
        DAY += days_up;
        mon--;
    }
    FormatDisplay(8,0,day);
}

/** 计算月份 */
void Month(){
    month = (MONTH + mon) % 12;
    if(month == 0){
        month = 12;
    }
    y = (MONTH + mon - 1) / 12;
    FormatDisplay(5,0,month);
    lcd.setCursor(7, 0);
    lcd.print('-');
```

```
/** 进入报警设置 */
void Set_Alarm(){
        if(digitalRead(add) == LOW &&
digitalRead(minus) == LOW){
        alarm_hour = hour;
        alarm_minute = minute;
        //alarm_choose  = 1;
        lcd.setCursor(0, 0);
        lcd.print("set alarm          ");
        lcd.setCursor(6, 1);
        lcd.print("00");            //闹钟秒数
        while(1){

            if(digitalRead(choose) == LOW)
{
                delay(ButtonDelay);
            if(digitalRead(choose) == LOW)
```

```
}


/** 计算年份 */
void Year(){
    year = ( YEAR + y ) % 9999;
    if(year == 0){
        year = 9999;
    }
    lcd.setCursor(0, 0);
    if(year < 1000){
        lcd.print("0");
    }
    if(year < 100){
        lcd.print("0");
    }
    if(year < 10){
        lcd.print("0");
```

```
{
                alarm_choose++;

            }
        }
        lcd.setCursor(9, 1);
        lcd.print(Temp_Alarm);
        lcd.setCursor(14, 1);
        lcd.print((char)223);       //显示 o
符号
        lcd.setCursor(15, 1);
        lcd.print("C");                  //显示
字母 C
        if(alarm_choose == 1){
            Set_Alarm_Hour();
        }else if(alarm_choose == 2){
            Set_Alarm_Minute();
```

```
    }
    lcd.print(year);
    lcd.setCursor(4, 0);
    lcd.print('-');
}

/** 根据年月日计算星期几 */
void Week(int y,int m, int d){
    if(m == 1){
        m = 13;
    }
    if(m == 2){
        m = 14;
    }
    int week = (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)%7+1;
    String weekstr = "";
```

```
        }else if(alarm_choose == 3){
            Set_Alarm_Temp();
        }else if(alarm_choose >= 4){
            alarm_choose = 0;
            break;
        }
    }
    delay(150);
  }
}

/** 正点蜂鸣 */
void Point_Time_Alarm(){
  if(minute == 0 && second == 0){
    tone(Tone,frequence);
    delay(500);
    noTone(Tone);
```

```
    switch(week){
        case 1: weekstr = "Mon. ";    break;
        case 2: weekstr = "Tues. ";   break;
        case 3: weekstr = "Wed. ";    break;
        case 4: weekstr = "Thur. ";   break;
        case 5: weekstr = "Fri. ";    break;
        case 6: weekstr = "Sat. ";    break;
        case 7: weekstr = "Sun. ";    break;
    }
lcd.setCursor(11, 0);
lcd.print(weekstr);
}

/** 显示时间、日期、星期 */
void Display() {
    time();
    Day();
```

```
    }
}

/** 闹钟 指定时间蜂鸣 */
void Clock_Alarm(){
    if(hour == alarm_hour && minute ==
alarm_minute && second == alarm_second){
        tone(Tone,frequence);
        delay(5000);
        noTone(Tone);
   }
}

/** 获取 LM35 温度 */
void GetTemperatures(){
        long a = analogRead(A3); // Get
temperatures
```

```
    Month();
    Year();
    Week(year,month,day);
}

/** 显示光标 */
void DisplayCursor(int rol, int row) {
    lcd.setCursor(rol, row);
    lcd.cursor();
    delay(100);
    lcd.noCursor();
    delay(100);
}

/** 设置初始时间 */
void set(int y, int mon, int d, int h, int m, int s){
    YEAR = y;
    MONTH = mon;
```

```
    Temperatures = 500.0*a/1023.0;
    lcd.setCursor(9, 1) ;
    lcd.print(Temperatures); //获取温度
    lcd.setCursor(14, 1);
    lcd.print((char)223); //显示 o 符号
    lcd.setCursor(15, 1);
    lcd.print("C"); //显示字母 C
}

/** 超过指定温度报警 */
void Temperatures_Alarm(){
    if(Temperatures >= Temp_Alarm){
        tone(Tone,frequence);
        delay(500);
        noTone(Tone);
    }
}
```

```
    DAY = d;
    HOUR = h;
    MINUTE = m;
    SECOND = s;
}
/** 通过按键设置时间 */
void Set_Time(int rol, int row, int &Time){
    DisplayCursor(rol, row);
    if(digitalRead(add) == LOW){
        delay(ButtonDelay);
        if(digitalRead(add) == LOW){
            Time ++;
        }
        Display();
    }
    if(digitalRead(minus) == LOW){
        delay(ButtonDelay);
```

```
void setup(){
    for(int i = 2;i <= 13; i++){
        pinMode(i,OUTPUT);
    }
    digitalWrite(add, HIGH);
    digitalWrite(minus, HIGH);
    digitalWrite(choose, HIGH);
    lcd.begin(16, 2);  //初始化 LCD1602
     //读取 DS1302 芯片的时间
     rtc.writeProtect(false);  // 关闭 DS1302
芯片写保护
     rtc.halt(false);              // 为 true 时
DS1302 暂停
    Time t;
    t=rtc.getTime();
set(t.year,t.mon,t.date,t.hour,t.min,t.sec);  // 设
```

```
        if(digitalRead(minus) == LOW){
            Time --;
        }
        Display();
    }
}

/** 按键选择 */
void Set_Clock(){
    if(digitalRead(choose)==LOW){

    }
```

```
置 DS1302 芯片初始时间
//    Time t;
//    t.hour=10;t.min=47;t.sec=59;
//    t.year=2025;t.mon=2;t.date=12;
//        rtc.setTime(t.hour,t.min,t.sec);  // 设 置
DS1302 芯片初始时间
//    rtc.setDate(t.year,t.mon,t.date);
//    set(t.year,t.mon,t.date,t.hour,t.min,t.sec);
    }

void loop() {
    seconds = millis()/1000;      //获取单片机
当前运行时间
    Display();          //显示时间
    Set_Clock();        //设置时间
```

	Set_Alarm(); //设置闹钟
	Point_Time_Alarm(); //正点蜂鸣
	Clock_Alarm(); //闹钟时间蜂鸣
	GetTemperatures(); //获取 LM35 温度
	Temperatures_Alarm(); //超过指定温度报警
	
	//将单片机的当前时间写到 DS1302 芯片中
	rtc.setTime(hour,minute,second);
	rtc.setDate(day,month,year);